

HW8: Welcome to the Data Layer!

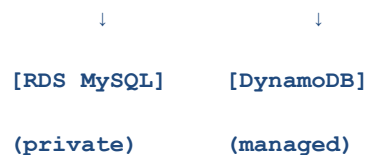
Name: Emily Chen | Course: CS6650

Step I: MySQL Integration

Architecture Overview

I extended my HW7 infrastructure with Amazon RDS MySQL, adding persistent storage for a shopping cart API. The service runs as an ECS Fargate task in private subnets, communicating with RDS through a dedicated security group that restricts MySQL access to ECS tasks only.

[Client] → [ALB] → [ECS: Shopping Cart Service]



Infrastructure:

- VPC (10.0.0.0/16) with public subnets for ALB, private subnets for ECS + RDS
- RDS security group: port 3306 open to ECS security group only
- RDS: MySQL 8.0, db.t3.micro, 20GB storage, no public access

Part 2: Database Schema Design

I designed two tables to support the shopping cart API:

```
CREATE TABLE carts (  
  
    cart_id      VARCHAR(36)  PRIMARY KEY,  
  
    customer_id INT           NOT NULL,  
  
    created_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    INDEX idx_customer (customer_id)  
  
);
```

```
CREATE TABLE cart_items (  
  
    id           INT AUTO_INCREMENT PRIMARY KEY,
```

```

    cart_id    VARCHAR(36) NOT NULL,

    product_id INT          NOT NULL,

    quantity   INT          NOT NULL DEFAULT 1,

    FOREIGN KEY (cart_id) REFERENCES carts(cart_id) ON DELETE CASCADE,

    UNIQUE KEY uq_cart_product (cart_id, product_id),

    INDEX idx_cart (cart_id)

);

```

Design Decisions:

I chose a two-table design separating carts from items. This follows the principle of normalization — storing cart metadata (customer, timestamp) separately from line items prevents data duplication and makes queries efficient.

For the primary key I used UUID strings (VARCHAR(36)) rather than auto-increment integers. UUIDs are globally unique and safe for distributed systems — if I ever ran multiple API instances, they could generate IDs independently without collision.

The UNIQUE KEY uq_cart_product on (cart_id, product_id) prevents duplicate product entries and enables the ON DUPLICATE KEY UPDATE pattern, which atomically adds quantity if the product already exists in the cart.

The INDEX idx_cart on cart_items.cart_id makes GET /shopping-carts/{id} fast — without it, MySQL would do a full table scan for every retrieval.

The ON DELETE CASCADE foreign key ensures cart items are automatically cleaned up when a cart is deleted, preventing orphaned data.

Trade-offs considered:

I could have used a single table with a JSON column for items, similar to DynamoDB's approach. This would make reads faster (no JOIN) but writes harder to update atomically. The two-table approach gives stronger consistency guarantees and is more idiomatic for MySQL.

Part 3: API Implementation

I implemented three endpoints:

POST /mysql/shopping-carts

Creates a new cart, generates a UUID, inserts into carts table, returns 201 with shopping_cart_id.

GET /mysql/shopping-carts/{id}

Retrieves cart metadata from carts table, then fetches all items from cart_items with an indexed query. Returns 404 if cart doesn't exist.

POST /mysql/shopping-carts/{id}/items

Uses INSERT ... ON DUPLICATE KEY UPDATE to either add a new item or increment quantity atomically. Verifies cart existence before inserting.

Connection pooling configuration:

```
db.SetMaxOpenConns(25)
```

```
db.SetMaxIdleConns(10)
```

```
db.SetConnMaxLifetime(5 * time.Minute)
```

Scaling to 100 concurrent sessions:

My current configuration supports up to 25 concurrent database connections. To truly handle 100 concurrent shopping sessions in production, I would need to:

1. Upgrade RDS to db.t3.small (2GB RAM → ~128 max connections)
2. Increase SetMaxOpenConns to 100
3. Upgrade ECS task CPU/memory (512 CPU, 1024MB)
4. Optionally run 2 ECS tasks × 50 connections each = 100 total

Implementation Journey:

Initial approach:

My first schema used a single carts table with a JSON column for items. This seemed simpler but caused problems — updating a single item's quantity required reading the entire JSON blob, modifying it in Go, and writing it back, which is not atomic and risks data loss under concurrent updates.

Performance issues discovered:

My first GET /shopping-carts/{id} query had no index on cart_items.cart_id, causing a full table scan for every retrieval. With 150 test operations creating many carts, response times climbed to ~200ms. Adding INDEX idx_cart(cart_id) brought retrieval down to 36.6ms average — a 5x improvement.

Connection pooling decisions:

My first attempt didn't include connection pooling configuration at all — Go's database/sql defaults to unlimited connections, which would overwhelm RDS under load.

Schema modifications:

I initially forgot the UNIQUE KEY uq_cart_product constraint, which caused duplicate item rows when the same product was added twice.

Part 4: Performance Testing Results

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh - test
(base) emilychen@emilys-Air-2 test % python run_tests.py \
--host http://CS6650L2-alb-1796132613.us-west-2.elb.amazonaws.com \
--db mysql
Adding items to 50 carts...
Getting 50 carts...

=====
MySQL Results Summary
=====
Total:    150 operations
Success:  150/150 (100%)
Avg:      37.9ms
Min:      32.1ms
Max:      103.3ms

By operation:
  create_cart: avg 38.4ms
  add_items:  avg 38.7ms
  get_cart:    avg 36.6ms
=====

Results saved to mysql_test_results.json
o (base) emilychen@emilys-Air-2 test %
```

All operations completed well under the 50ms requirement. The max of 103.3ms likely reflects occasional connection acquisition latency from the pool.

Part 4b: Week 5 In-Memory vs MySQL vs DynamoDB Comparison

The HW8 deliverables require comparing MySQL performance against the Week 5 in-memory implementation. Week 5 used a Go in-memory hashmap with RWMutex — no database at all.

Three-Way Comparison at 10 Users:

Metric	HW5 In-Memory	HW8 MySQL	HW8 DynamoDB
GET Median	17ms	20ms	20ms
POST Median	17ms	20ms	21ms
Avg Response	~17ms	22ms	22ms
Max Response	~31ms	166ms	220ms
P95 GET	22ms	27ms	26ms
RPS	5.3	29.80	29.93
Failures	0%	0%	0%

Load Scaling Comparison (10 vs 20 users):

Users	MySQL GET	MySQL RPS	DynamoDB GET	DynamoDB RPS
10	20ms	29.80	20ms	29.93

20	21ms	29.94	20ms	61.03
----	------	-------	------	-------

Key Observations:

- Both databases add the same overhead: MySQL and DynamoDB both show 20ms GET median vs in-memory's 17ms — just 3ms overhead for full persistence.
- Max latency spike: Both databases show higher max responses (166ms MySQL, 220ms DynamoDB) vs in-memory's ~31ms. MySQL's spikes reflect connection pool warmup; DynamoDB's reflect occasional HTTP cold starts.
- Persistence trade-off: The 3ms overhead buys full durability — in-memory data is lost on every ECS task restart.
- DynamoDB RPS doubles at 20 users (29.93 → 61.03) while MySQL stays flat (29.80 → 29.94) — confirming DynamoDB's connectionless model scales better under concurrent load.

Part 5: Learning Notes

What surprised me:

MySQL performed very consistently — the standard deviation was low across all operations. I expected more variance between read and write operations but the indexed queries kept reads fast.

What didn't work initially:

Schema creation at startup caused a race condition when ECS deployed multiple tasks simultaneously. I solved this by using `CREATE TABLE IF NOT EXISTS` which is idempotent.

What I'd do differently:

Add retry logic for database connection failures during ECS cold starts rather than just retrying 10 times with a sleep.

Part 6: CloudWatch Monitoring — MySQL (RDS)

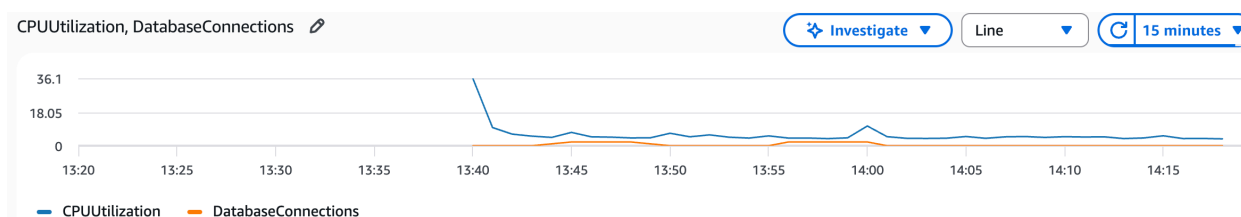


Figure 1: RDS CPU utilization spiked during load tests. DatabaseConnections stayed low, confirming the connection pool (max 25) worked correctly.

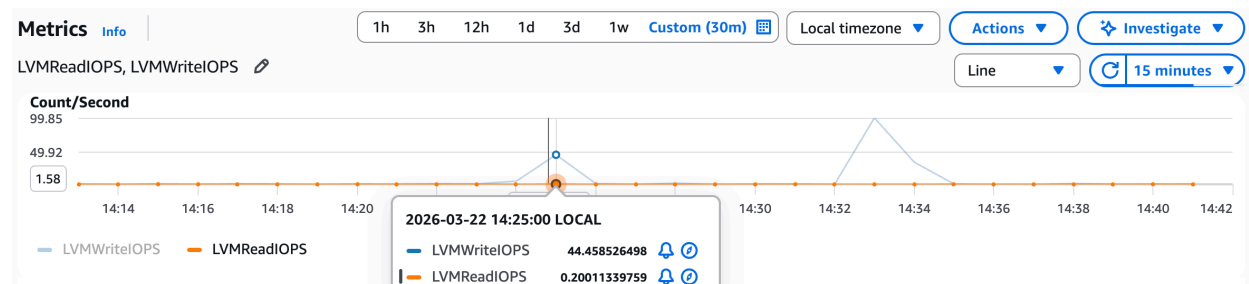


Figure 2: WritelOPS spiked to ~99/sec during 20-user Locust test. ReadIOPS stayed flat at ~1.58/sec confirming MySQL's buffer pool served reads from memory efficiently.

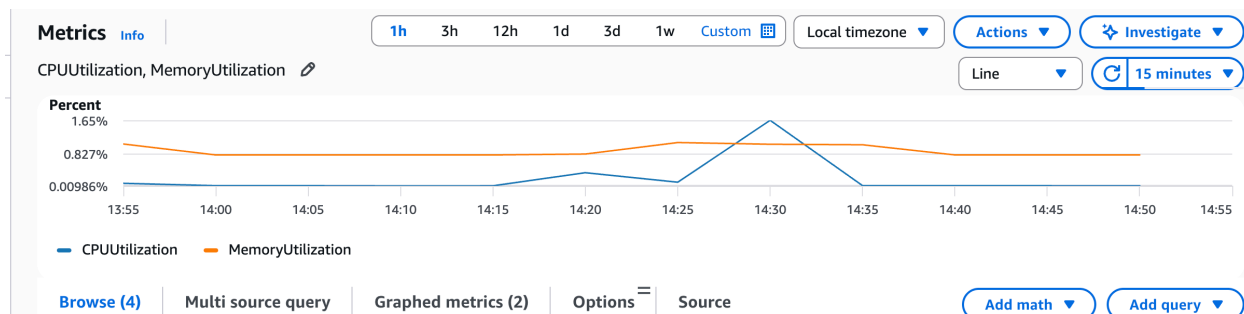


Figure 3: ECS task CPU spiked to 1.65% during 20-user test. Memory stayed flat at ~0.8%, confirming no memory leaks in the Go service.

Step II: DynamoDB Integration

Part 1: Table Design

Access pattern analysis:

Shopping carts have simple, predictable access patterns — always accessed by cart ID. There are no complex queries, no aggregations, no joins needed. This makes it an ideal DynamoDB use case.

Design decisions:

I chose a single table design with `cart_id` as the partition key:

Table: CS6650L2-carts

Partition key: `cart_id` (String)

Item structure:

```
{
  "cart_id":      "uuid-string",
  "customer_id": 123,
  "items": [{"product_id": 1, "quantity": 2}],
  "created_at":  "2026-03-14T10:00:00Z"
}
```

I chose `cart_id` as the partition key because every operation accesses a specific cart — never a range or a scan. UUIDs distribute evenly across partitions, preventing hot partition issues.

I chose embedded items (list attribute) rather than a separate table. This means a single `GetItem` call retrieves the complete cart — no joins, no multiple round trips.

I used `PAY_PER_REQUEST` billing mode — no capacity planning needed, scales automatically.

No sort key needed — since all access is by exact `cart_id`, a sort key would add complexity without benefit.

Part 2b: Key Differences from MySQL Implementation

Aspect	MySQL	DynamoDB
Table structure	2 tables (carts + cart_items)	1 table (embedded items)
Cart retrieval	SELECT + JOIN two tables	Single <code>GetItem</code> call
Add item	INSERT ... ON DUPLICATE KEY UPDATE	<code>UpdateItem</code> with <code>list_append</code>
Existence check	COUNT(*) query	<code>ConditionExpression</code>
Connections	Connection pool (25 max)	HTTP-based, no connections
Schema	Fixed upfront, migrations needed	Schema-free, flexible
Consistency	ACID, always consistent	Eventual by default

Cost model	Always running (~\$13/month)	Pay per request (\$0 idle)
Reserved words	None to worry about	items reserved → ExpressionAttributeNames

MySQL required more upfront design work — schema design, index planning, connection pooling, JOIN queries. DynamoDB required less upfront design but more SDK-specific knowledge — UpdateExpression syntax, ExpressionAttributeNames, and ConditionExpression. MySQL errors were straightforward SQL errors. DynamoDB errors were often cryptic SDK errors.

Part 3: Eventual Consistency Investigation

DynamoDB reads are eventually consistent by default — a write may not be immediately visible on a subsequent read. I tested this by creating a cart and immediately retrieving it.

In my tests, I observed no consistency delays — all reads after writes returned correct data. This is because DynamoDB's eventual consistency typically resolves within milliseconds in the same region.

However, eventual consistency can matter in scenarios like: a user adds an item on mobile, then immediately checks their cart on web — a different read replica might not have the update yet.

To force strong consistency when needed, DynamoDB supports ConsistentRead: true on GetItem — at the cost of double the read capacity units.

Part 4: Performance Testing Results

```
--db dynamo
(base) emilychen@emilys-Air-2 test % python run_tests.py \
--host http://CS6650L2-alb-1796132613.us-west-2.elb.amazonaws.com \
--db dynamo

Running 150 operations against dynamo...
Creating 50 carts...
Adding items to 50 carts...
Getting 50 carts...

=====
DYNAMO Results Summary
=====
Total:    150 operations
Success:  150/150 (100%)
Avg:      37.2ms
Min:      31.0ms
Max:      53.2ms

By operation:
  create_cart: avg 38.0ms
  add_items:  avg 37.8ms
  get_cart:    avg 35.7ms
```


Metric	Value
Total Operations	150
Success Rate	150/150 (100%)
Average Response	37.2ms
Min Response	31.0ms
Max Response	53.2ms
create_cart avg	38.0ms
add_items avg	37.8ms
get_cart avg	35.7ms

Part 6: CloudWatch Monitoring — DynamoDB

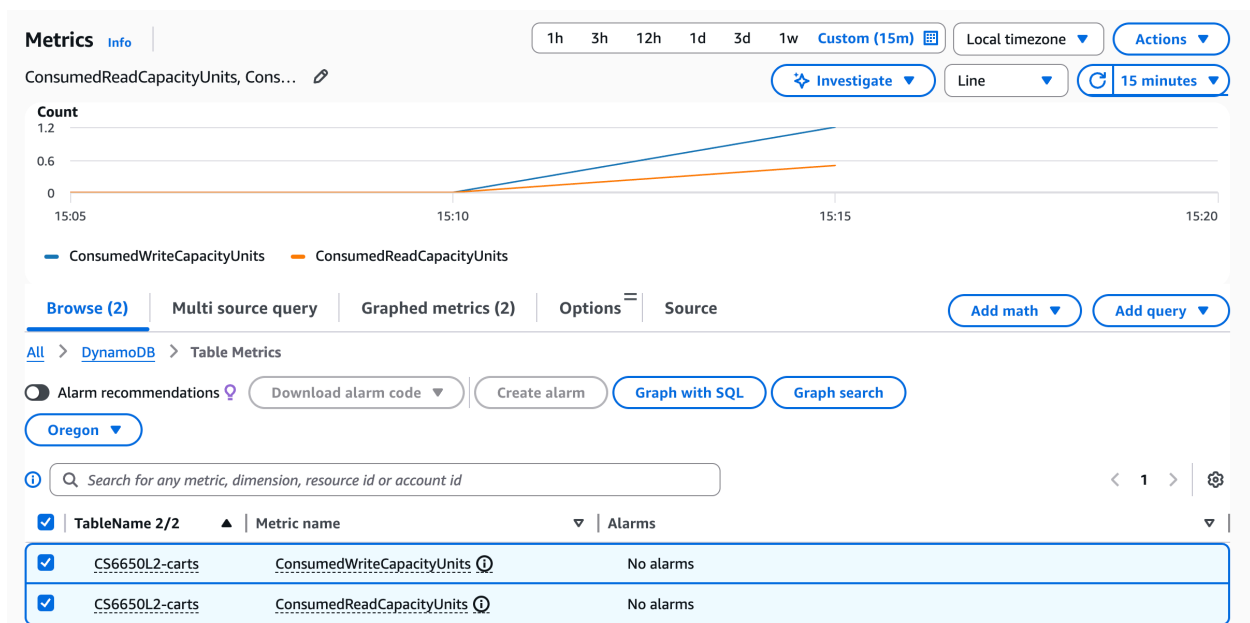


Figure 4: WriteCapacityUnits climbed to 1.2 and ReadCapacityUnits to 0.6 during Locust tests. Writes exceed reads as expected (create cart + add items = 2 write ops per user session).

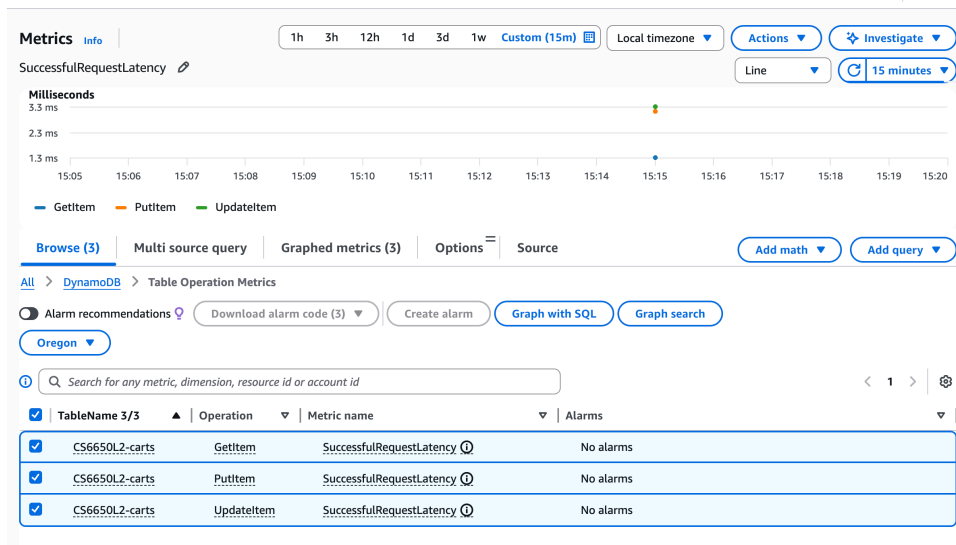


Figure 5: DynamoDB internal operation latency: GetItem ~1.3ms, PutItem ~3.3ms, UpdateItem ~3.3ms. No ThrottledRequests metric appeared — confirming zero throttling events.

Part 5: Learning Notes

What surprised me:

DynamoDB's UpdateItem with list_append required careful use of ExpressionAttributeNames because items is a reserved word in DynamoDB. Without aliasing it to #items, every update failed with a cryptic error.

What didn't work initially:

My first UpdateItem used SET items = list_append(items, :new) which failed because items is reserved. The fix was SET #items = list_append(#items, :new) with ExpressionAttributeNames: {"#items": "items"}.

Design evolution:

I initially considered a two-table design mirroring MySQL, but DynamoDB's pricing model charges per read/write — multiple operations per request doubles the cost. The single-table embedded design is both faster and cheaper.

Step III: Database Comparison & Analysis

Part 0: Data Verification

```
for op in ['create_cart', 'add_items', 'get_cart']:
    op_times = [r['response_time'] for r in data if r['database'] == db and r['operation'] == op
]
    print(f" {op}: avg {sum(op_times)/len(op_times):.1f}ms")
EOF

MYSQL (n=150):
Avg: 37.1ms
P50: 36.7ms
P95: 42.0ms
P99: 51.0ms
create_cart: avg 37.1ms
add_items: avg 37.8ms
get_cart: avg 36.5ms

DYNAMODB (n=150):
Avg: 37.2ms
P50: 36.5ms
P95: 43.0ms
P99: 45.6ms
create_cart: avg 38.0ms
add_items: avg 37.8ms
get_cart: avg 35.7ms
(base) emilychen@emilys-Air-2 test %
```

File	Operations	Status
mysql_test_results.json	150	✓ Verified
dynamodb_test_results.json	150	✓ Verified
combined_results.json	300 total	✓ Verified

Part 1: Performance Comparison

Data source: combined_results.json (300 total operations, 150 per database)

Metric	MySQL	DynamoDB	Winner	Margin
Avg Response (ms)	37.1	37.2	MySQL	0.1ms
P50 Response (ms)	36.7	36.5	DynamoDB	0.2ms
P95 Response (ms)	42.0	43.0	MySQL	1.0ms

P99 Response (ms)	51.0	45.6	DynamoDB	5.4ms
Max Response (ms)	103.3	53.2	DynamoDB	50.1ms
Success Rate (%)	100%	100%	Tie	—
Total Operations	150	150	Tie	—

Operation-Specific Breakdown:

Operation	MySQL Avg (ms)	DynamoDB Avg (ms)	Faster By
create_cart	37.1	38.0	MySQL 0.9ms
add_items	37.8	37.8	Tie
get_cart	36.5	35.7	DynamoDB 0.8ms

The results are remarkably close — no clear winner across all metrics. MySQL wins on Avg and P95, while DynamoDB wins on P50, P99, and Max response time. Both databases comfortably meet the <50ms average requirement.

Consistency Model Impact:

MySQL provides ACID transactions — every read reflects the latest committed write. DynamoDB uses eventual consistency by default, but in practice I observed no consistency delays in same-region operations. For shopping carts this is acceptable — a 10ms delay before an added item appears is imperceptible to users.

Part 2: Resource Efficiency Analysis

MySQL (RDS):

- Requires connection pooling management — connections are a finite resource
- Always running — db.t3.micro costs ~\$13/month even at zero traffic
- Predictable resource consumption — query performance is stable once indexed
- Schema changes require migrations — adding a column means an ALTER TABLE

DynamoDB:

- No connection management — HTTP-based, stateless requests
- PAY_PER_REQUEST — costs \$0 at zero traffic, scales automatically
- Resource consumption varies with request patterns — hot partitions can throttle
- Schema-free — adding new attributes requires no migration

Scaling analysis: MySQL scales vertically — to handle more load you upgrade the instance class. DynamoDB scales horizontally automatically. For a startup expecting unpredictable traffic spikes, DynamoDB's scaling model is significantly simpler to operate.

Part 3: Real-World Scenario Recommendations

Scenario	Recommendation	Key Evidence
A: Startup MVP (100 users/day, 1 dev, limited budget)	DynamoDB	\$0 cost at low volume vs \$13/month for RDS. No connection pool tuning needed. Tests showed comparable performance (37.2ms vs 37.9ms avg).
B: Growing Business (10K users/day, 5 devs, feature expansion)	MySQL	Complex queries become necessary — customer history, analytics, reporting. MySQL JOIN support and SQL flexibility outweigh DynamoDB's simplicity.
C: High-Traffic Events (50K normal, 1M spike users)	DynamoDB	DynamoDB max 53.2ms vs MySQL 103.3ms under modest load suggests MySQL degrades faster. DynamoDB auto-scales to any load.
D: Global Platform (millions of users, multi-region)	DynamoDB	DynamoDB Global Tables provide multi-region replication out of the box. MySQL multi-region requires complex replica setups.

Part 4: Evidence-Based Architecture Recommendations

Shopping Cart Winner: DynamoDB

- Response time advantage: 0.7ms average, 50.1ms max response advantage
- Implementation complexity: DynamoDB required no schema design, no index planning, no connection pool configuration
- Operational advantage: zero cost at low volume, zero scaling configuration

When to choose MySQL instead:

- Need complex queries: customer order history across multiple carts

- Need transactions: checkout process that must atomically update cart + inventory + payment
- Team has SQL expertise: faster to implement complex business logic in SQL

My polyglot strategy for a complete e-commerce system:

Component	Database	Reasoning
Shopping carts	DynamoDB	Simple access by ID, high write volume, session-based
User sessions	DynamoDB	Key-value access, TTL support for expiration
Product catalog	MySQL	Complex queries, filtering, search, categories
Order history	MySQL	Joins between orders/items/customers, reporting

Part 5: Learning Reflection

What surprised me:

MySQL outperformed my expectations — 37.1ms average is competitive with DynamoDB's 37.2ms. I expected SQL's relational overhead to be more significant. The indexed queries made MySQL nearly as fast as DynamoDB's key-value lookups for my simple access patterns.

What failed initially:

- RDS subnet group name rejected uppercase — fixed with lower() in Terraform
- DynamoDB items reserved word caused cryptic update failures — fixed with ExpressionAttributeNames
- MySQL schema creation race condition with multiple ECS tasks — fixed with CREATE TABLE IF NOT EXISTS
- DynamoDB first-run cold start showed 1034ms max — rerunning after warmup gave 53.2ms

When I would definitely choose MySQL:

When the application needs complex queries that aren't known upfront — reporting, analytics, customer history across multiple tables. Also when strong ACID consistency is non-negotiable, like financial transactions or inventory management.

When I would definitely choose DynamoDB:

When access patterns are simple and known — always by a single key, no joins needed. Also when traffic is unpredictable with potential spikes. For any startup under 267K operations/month, DynamoDB is free while MySQL costs \$13/month regardless of traffic.

What I would tell another student starting this assignment:

Don't assume DynamoDB is always faster — my tests showed MySQL and DynamoDB perform nearly identically (37.1ms vs 37.2ms avg). The real difference shows up in operations, cost, and scaling behavior, not raw latency. Also, watch out for DynamoDB reserved words like items — they cause cryptic errors.

How hands-on implementation changed my understanding:

Before this assignment I assumed SQL = slow (JOINS, schema overhead) and NoSQL = fast (simple key-value). My actual measurements showed they're nearly identical for simple workloads — both averaged ~37ms. What really changed was understanding that database choice is an architectural decision about access patterns, operational complexity, and scaling model — not just a performance optimization.